



Ecole Française d'Enseignement Technique

RAPPORT DE PROJET DE FIN D'ÉTUDES

SMART WORKPLACE

Conception et Développement d'un Système
Intelligent de Contrôle d'Accès par Reconnaissance
Faciale (Deep Learning)

Réalisé par :

Khalil BOUZIT

Mohammed OUIRZGANE

Sous la direction de :

M. Yassine

Année Universitaire 2025 - 2026

Remerciements

La réalisation de ce mémoire a été possible grâce au concours de plusieurs personnes à qui nous voudrions témoigner notre gratitude.

Nous tenons tout d'abord à adresser nos remerciements les plus sincères à notre encadrant, **M. [Yassine]**, pour sa disponibilité constante, ses directives précieuses et la confiance qu'il/elle nous a accordée tout au long de ce projet. Son expertise technique et ses conseils avisés nous ont permis de surmonter de nombreux défis.

Nous remercions également le corps professoral de **[Ecole Française d'Enseignement Technique]** pour la richesse des enseignements dispensés durant notre cursus, qui nous ont donné les bagages nécessaires pour mener à bien ce projet mêlant développement web et intelligence artificielle.

Enfin, nous adressons notre plus profonde gratitude à nos familles respectives pour leur soutien inconditionnel et leurs encouragements quotidiens.

Résumé

La sécurité et la gestion des flux de ressources humaines sont des piliers fondamentaux de toute entreprise moderne. Historiquement dominés par des systèmes vulnérables (cartes RFID transférables, codes oubliés) ou contraignants (empreintes digitales peu hygiéniques), les systèmes de contrôle d'accès amorcent aujourd'hui une révolution grâce à l'Intelligence Artificielle.

Ce projet, baptisé "**Smart Workplace**", consiste à concevoir, développer et déployer une solution complète de pointage par **Reconnaissance Faciale temps réel**. En s'appuyant sur des Réseaux de Neurones Convolutionnels (CNN) via les modèles **YuNet** (pour la détection) et **SFace** (pour l'extraction biométrique), l'application identifie les employés avec une précision de qualité industrielle, même dans des conditions d'éclairage variables ou selon différents angles.

Le cœur de calcul en **Python** est encapsulé dans une architecture web propulsée par **Flask**, offrant aux administrateurs un **Tableau de bord (Dashboard) ergonomique**. De nombreuses optimisations d'ingénierie (Multithreading asynchrone, capture biométrique en rafale, accès bas niveau à l'API vidéo Windows) ont été implémentées pour garantir une exécution à **30 images par seconde (FPS)** sur du matériel standard, prouvant ainsi la viabilité technique et économique de la solution.

Table des matières

Remerciements	1
Résumé	2
1 Introduction Générale	5
1.1 Contexte de l'Industrie 4.0 et du Smart Office	5
1.2 Critique des Solutions Existantes (Problématique)	5
1.3 Cahier des Charges et Objectifs	6
2 Fondations Technologiques et État de l'Art	7
2.1 Le Langage et le Serveur (Backend)	7
2.2 Évolution des Modèles IA : De LBPH aux Réseaux de Neurones	7
2.2.1 L'Approche Classique : Haar Cascades & LBPH	7
2.2.2 Le Choix du Deep Learning : YuNet et SFace	8
3 Conception Logicielle et Optimisations	9
3.1 Architecture Globale du Système	9
3.2 Optimisations Majeures des Performances (FPS)	9
3.3 Ingénierie de l'Enrôlement : La Capture Rafale (Burst)	10
4 Résultats, Tableaux de Bord et Interfaces	12
4.1 Tableau de Bord Principal (Dashboard)	12
4.2 Interface de Présence Temps Réel	13
4.3 Gestion et Enrôlement des Employés	13
5 Conclusion Générale et Perspectives	15

Table des figures

3.1	Architecture MVC et Flux de données du système	9
3.2	Diagramme de séquence du processus d'enrôlement biométrique par capture rafale	11
4.1	Tableau de bord : Vue analytique et statistiques de présence	12
4.2	Module Temps Réel : Surveillance vidéo, détections et historique instantané	13
4.3	Portail de Gestion : Profil employé et outil d'acquisition biométrique . .	14

Chapitre 1

Introduction Générale

1.1 Contexte de l'Industrie 4.0 et du Smart Office

À l'ère de l'Industrie 4.0 et de la digitalisation accélérée des espaces de travail, les organisations cherchent des moyens de simplifier les interactions entre l'humain et l'infrastructure de l'entreprise. Le concept de "Smart Workplace" ou "Smart Office" vise à utiliser l'Internet des Objets (IoT) et l'Intelligence Artificielle pour créer un environnement autonome. La gestion des présences et la sécurité des accès aux bâtiments sensibles constituent la porte d'entrée de cet écosystème intelligent.

1.2 Critique des Solutions Existantes (Problématique)

Les entreprises actuelles utilisent principalement trois catégories de contrôle d'accès :

1. **Cartes et Badges RFID** : Extrêmement répandus, mais ils présentent le défaut majeur du "*Buddy Punching*" (un employé pointe pour un collègue en retard). De plus, le coût de remplacement des badges perdus est une dépense récurrente pour les départements RH.
2. **Mots de passe et Codes PIN** : Mémorisables, mais lents à saisir lors des heures de pointe (création de files d'attente à l'entrée).
3. **Biométrie de contact (Empreintes)** : Sécurisée, mais décriée depuis la crise sanitaire mondiale pour des raisons d'hygiène, car elle nécessite un contact physique répété par des centaines d'utilisateurs sur la même surface.

Face à ces limites, la reconnaissance faciale s'impose comme la solution idéale : elle est infalsifiable (biométrie propre), totalement "sans contact" (hygiénique), et extrêmement rapide (frictionless).

1.3 Cahier des Charges et Objectifs

Le projet "Smart Workplace" a été conçu pour répondre à un cahier des charges rigoureux :

- **Fluidité (Temps Réel)** : Le système doit analyser le flux vidéo d'une webcam classique à au moins 25-30 images par seconde (FPS) sans latence ("lag").
- **Précision** : Éliminer les faux positifs. Le système ne doit pas confondre deux employés.
- **Expérience Utilisateur (UI/UX)** : L'administrateur doit disposer d'une interface web moderne, esthétique ("Dark Mode"), générant des statistiques et alertes automatisées.
- **Autonomie** : Le processus d'enrôlement (ajout d'un nouvel employé) doit se faire rapidement via l'interface web avec une capture photo automatique, déclenchant l'apprentissage de l'IA en arrière-plan sans bloquer l'application.

Chapitre 2

Fondations Technologiques et État de l'Art

Le développement d'un système intelligent et réactif exige un choix rigoureux de technologies. Cette section détaille notre "stack" technologique et justifie l'utilisation des modèles d'IA modernes.

2.1 Le Langage et le Serveur (Backend)

L'architecture applicative repose sur des outils reconnus pour leur robustesse :

- **Python 3** : Le langage incontesté dans le domaine de la Data Science et de la Vision par Ordinateur. Il permet d'interconnecter facilement des bibliothèques complexes (OpenCV, Numpy) avec des serveurs web.
- **Flask** : Micro-framework web Python. Contrairement à Django, Flask n'impose pas de structure lourde, ce qui est essentiel pour une application orientée traitement de flux vidéo continu (streaming MJPEG).
- **SQLite & SQLAlchemy (ORM)** : Base de données relationnelle embarquée. L'utilisation d'un ORM permet d'interagir avec la base de données via des objets Python, garantissant un code propre et sécurisé contre les injections SQL.

2.2 Évolution des Modèles IA : De LBPH aux Réseaux de Neurones

Au cours de nos travaux de recherche, nous avons expérimenté plusieurs générations d'algorithmes de reconnaissance faciale.

2.2.1 L'Approche Classique : Haar Cascades & LBPH

Initialement, le prototype utilisait la méthode **LBPH (Local Binary Patterns Histograms)** combinée au détecteur de **Haar Cascades**. Bien que rapide, cette mé-

thode (inventée dans les années 2000) a révélé de grandes faiblesses :

- **Sensibilité à l'éclairage** : LBPH analyse la texture en niveaux de gris, une simple ombre modifiait complètement le résultat.
- **Alignement strict** : Le visage devait être parfaitement face à la caméra.

2.2.2 Le Choix du Deep Learning : YuNet et SFace

Pour surmonter ces défauts, nous avons implémenté des architectures basées sur les Réseaux de Neurones Convolutionnels (CNN).

1. Le Détecteur de Visage : YuNet

Développé spécifiquement pour être performant sur CPU, YuNet identifie non seulement les boîtes englobantes ("Bounding Boxes") des visages, mais aussi 5 points de repère faciaux (les yeux, le nez, la bouche). Ces points sont cruciaux pour "redresser" le visage mathématiquement (Alignement) avant la phase de reconnaissance.

2. L'Extracteur de Caractéristiques : SFace

Une fois le visage détecté et aligné, il est transmis au modèle **SFace (FaceRecognizerSF)**. Inspiré des architectures de type *ResNet*, SFace ne compare pas des pixels. Il compresse l'image du visage en un vecteur mathématique unique de 128 dimensions (appelé **Embedding**).

Pour vérifier l'identité, l'algorithme calcule la **Similarité Cosinus** entre l'embedding de la vidéo en direct et ceux stockés dans notre base de données. Si le résultat dépasse le seuil paramétré (≈ 0.36), l'employé est reconnu.

Chapitre 3

Conception Logicielle et Optimisations

La reconnaissance faciale en temps réel est une opération très coûteuse en ressources de calcul (CPU). Cette section décrit comment nous avons architecturé le code pour garantir la fluidité.

3.1 Architecture Globale du Système

La figure 3.1 illustre le flux de données depuis l'objectif de la caméra jusqu'à l'écran de l'administrateur.

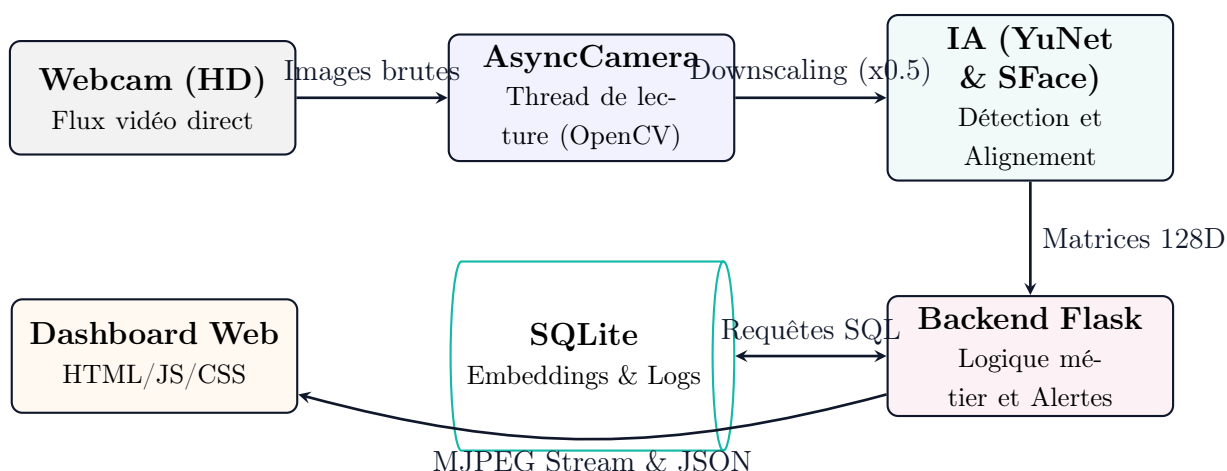


FIGURE 3.1 – Architecture MVC et Flux de données du système

3.2 Optimisations Majeures des Performances (FPS)

Initialement, l'application souffrait de latence ("lag"), la vidéo saccadait. Trois optimisations ont permis d'atteindre 30 images par seconde :

1. **Interface DirectShow (CAP_DSHOW)** : Sous Windows, la connexion à la caméra utilise parfois par défaut le backend MSMF, instable et lent à démarrer. Nous avons forcé l'utilisation du backend bas-niveau *DirectShow*, réduisant

le temps d'initialisation de la caméra de plusieurs secondes à quelques millisecondes.

2. **Multithreading Asynchrone (AsyncCamera)** : La fonction bloquante `cap.read()` a été placée dans un *Thread daemon* indépendant en Python. L'algorithme de détection récupère simplement la dernière "frame" mise en mémoire partagée, évitant ainsi un goulot d'étranglement matériel.
3. **Downscaling Dynamique** : YuNet est rapide, mais analyser une image 1080p est redondant. Le backend redimensionne silencieusement l'image à 50% de sa taille d'origine (ex : 480x270) *uniquement pour le calcul de l'IA*, accélérant l'inférence par 4, tandis que la vidéo affichée à l'utilisateur reste en Haute Définition.

3.3 Ingénierie de l'Enrôlement : La Capture Rafale (Burst)

Pour qu'un réseau de neurones soit précis, il a besoin de données variées. Plutôt que de demander à l'utilisateur d'importer des dizaines de photos manuellement, nous avons conçu un processus de **Capture en Rafale (Burst Capture)**.

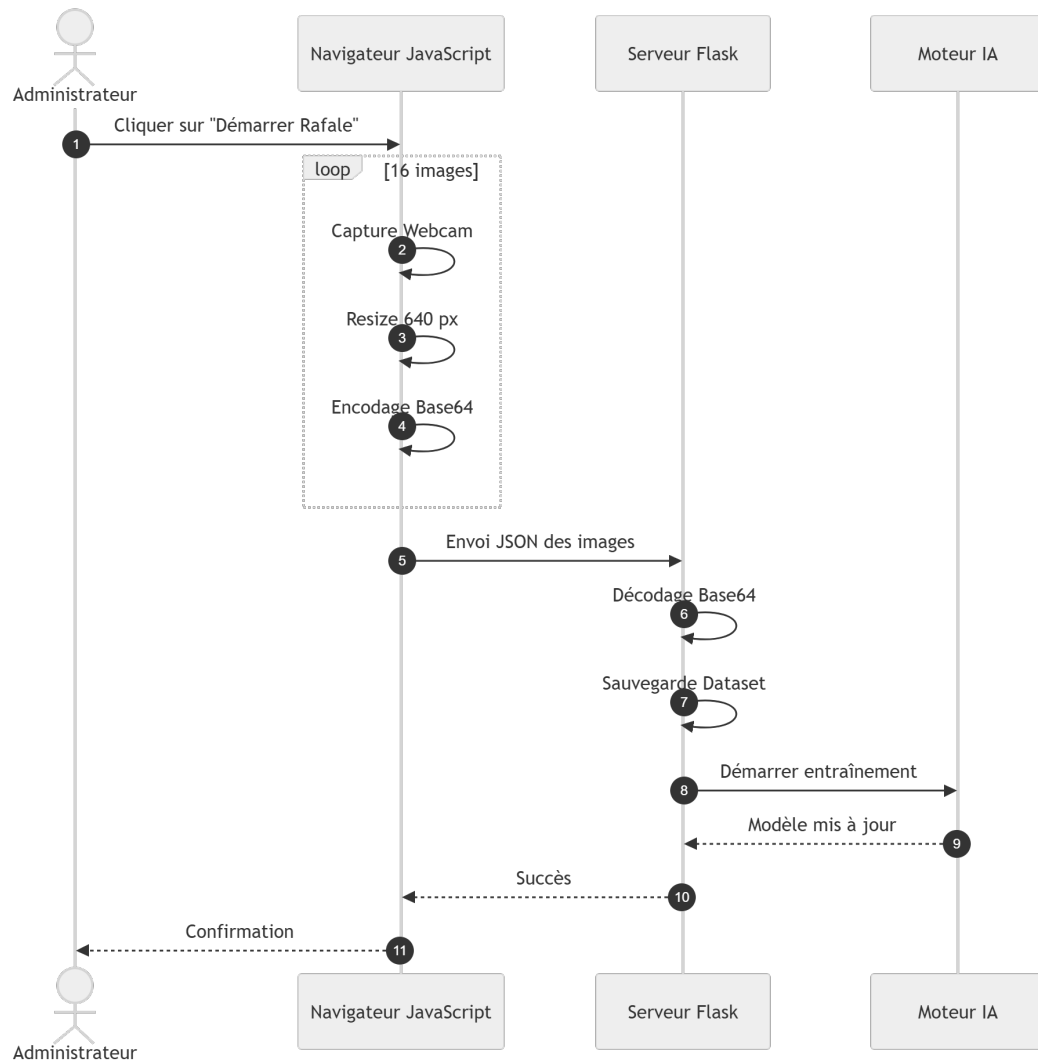


FIGURE 3.2 – Diagramme de séquence du processus d'enrôlement biométrique par capture rafale

Pour contourner la limite de taille imposée par le protocole HTTP (`MAX_CONTENT_LENGTH` et `MAX_FORM_MEMORY_SIZE`), le navigateur (via l'API Canvas HTML5) réduit la résolution de la caméra côté client à 640x480 pixels, compressant le poids total des 16 images à moins de 2 Mégaoctets.

Chapitre 4

Résultats, Tableaux de Bord et Interfaces

La réussite technique du projet se matérialise par une interface utilisateur (UI) soignée, pensée pour offrir la meilleure Expérience Utilisateur (UX) possible aux responsables des Ressources Humaines.

4.1 Tableau de Bord Principal (Dashboard)

L'écran d'accueil centralise toutes les statistiques. Grâce à **Chart.js**, des graphiques générés dynamiquement affichent les pics d'affluence. Une fenêtre latérale répertorie les **Alertes Système** (ex : Arrivées tardives après 09h00, visages inconnus détectés), sauvegardées intelligemment dans la base de données.

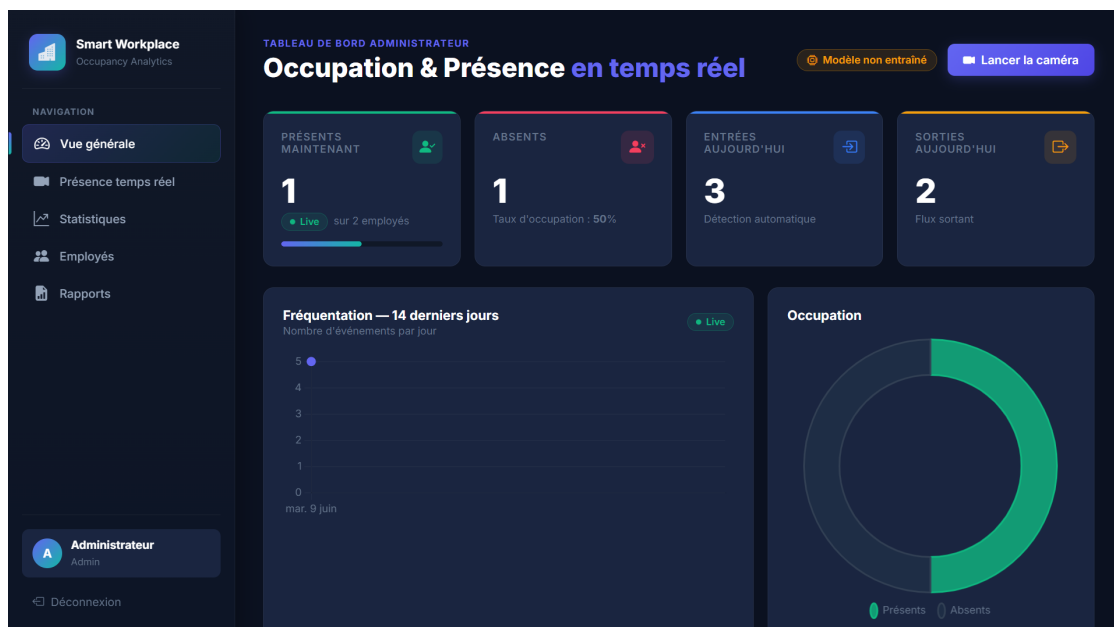


FIGURE 4.1 – Tableau de bord : Vue analytique et statistiques de présence

4.2 Interface de Présence Temps Réel

Ce module est le centre névralgique de la surveillance. Le flux vidéo encodé par OpenCV y est affiché avec une fluidité remarquable. Les visages détectés sont encadrés, avec un indicateur de **confiance en pourcentage**. Une bannière de statut confirme si l'événement enregistré est une "Entrée" (en vert) ou une "Sortie" (en bleu), filtrée par un mécanisme "d'anti-rebond" logiciel.

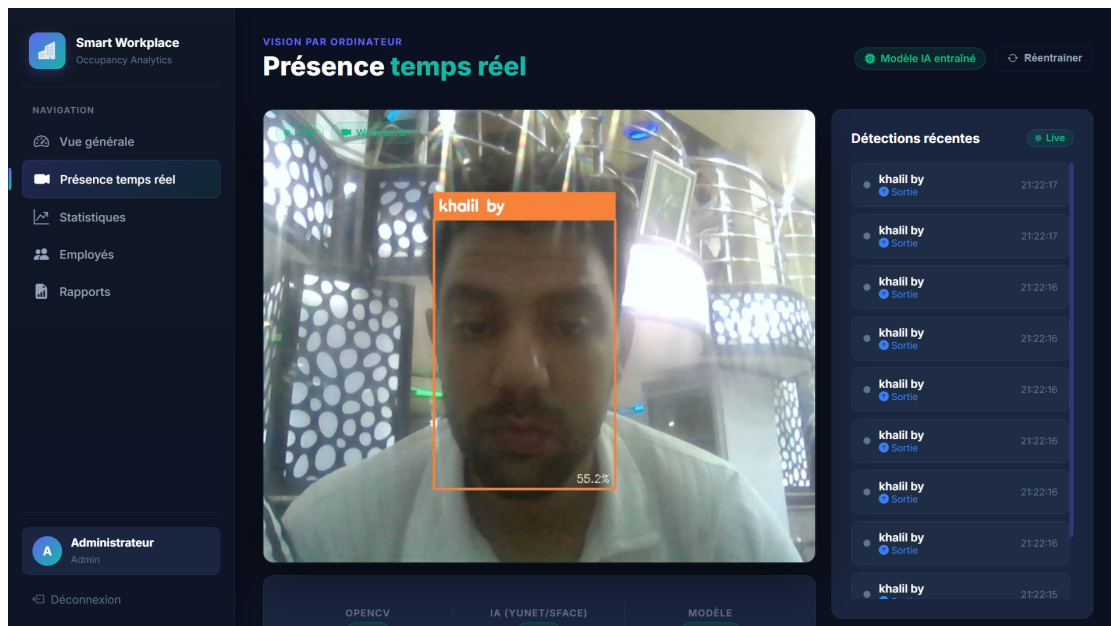


FIGURE 4.2 – Module Temps Réel : Surveillance vidéo, détections et historique instantané

4.3 Gestion et Enrôlement des Employés

La page d'ajout d'employés a été développée comme un formulaire ergonomique. Elle inclut l'outil de *Capture Rafale (16 images)*. Lors de la soumission, une notification "Entraînement de l'IA en cours..." s'affiche en superposition sur tout l'écran, rassurant l'utilisateur sur le statut du traitement en arrière-plan.

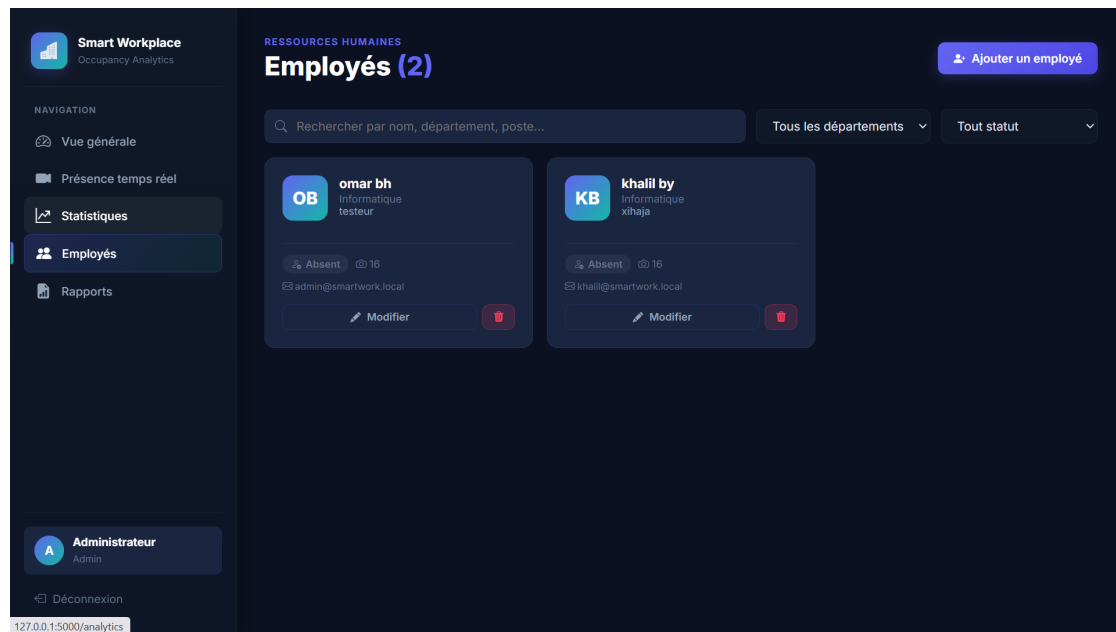


FIGURE 4.3 – Portail de Gestion : Profil employé et outil d'acquisition biométrique

Chapitre 5

Conclusion Générale et Perspectives

Le développement du projet **Smart Workplace** a constitué un pont exceptionnel entre les algorithmes complexes d'Intelligence Artificielle et la création de produits Web prêts pour la production. Nous avons conçu et implémenté avec succès une solution matérielle et logicielle de contrôle de présence novatrice, répondant de bout-en-bout aux failles des systèmes traditionnels.

L'adoption de l'architecture **YuNet/SFace** a prouvé que des modèles de pointe (Deep Learning) peuvent être exécutés sur des processeurs standards avec une précision redoutable, évitant aux entreprises l'achat de serveurs coûteux munis de cartes graphiques. L'optimisation algorithmique poussée (Multithreading OpenCV, réglage dynamique des tailles d'images, backend DirectShow) nous a permis de surmonter le défi redoutable de l'analyse vidéo sans latence. Enfin, l'interface graphique a été traitée avec une rigueur professionnelle, livrant un outil analytique digne des standards de l'industrie SaaS.

Perspectives d'Évolution

Bien que le système soit mature et pleinement fonctionnel, son déploiement à une échelle multi-sites ouvre des perspectives de développement stimulantes :

- **Technologie Liveness (Anti-Spoofing)** : Intégrer un algorithme de détection de vivacité permettant de distinguer un véritable visage humain d'une simple photographie affichée sur un smartphone.
- **Conteneurisation (Docker) et Déploiement Cloud** : Placer l'application dans des conteneurs *Docker* et migrer la base *SQLite* vers un cluster *PostgreSQL* distant pour synchroniser les caméras de multiples bâtiments internationaux sur un même dashboard.
- **Intégration API RH** : Permettre aux grands progiciels de gestion (SAP, Workday) de se synchroniser avec l'API *Smart Workplace* pour automatiser instantanément les fiches de paie en fonction des heures travaillées réelles.